



Enterprise OCI Network Design: Architecture, Load Balancing, and Terraform Implementation

Ali Yousef

Week 2 Documentation

Compartment: intern-02-ali-youssef-cmp

Region: me-jeddah-1 (Saudi Arabia West – Jeddah)

Summary

In this documentation, I cover a **defense-in-depth network** I built for a two-tier application on Oracle Cloud Infrastructure (OCI), and verified end to end with Terraform. Production workloads should *never* expose an application server or a database directly to the internet, and I still needed a managed, audited path to those private resources for maintenance. I built this deployment to satisfy both requirements, and in this write-up I set out the infrastructure-as-code patterns I used to keep the configuration reusable and safe to change over time.

I cover two things: the OCI networking concepts behind my design (load balancing, tiering, private-access options), and the Terraform mechanics I used to build it, including typed variables and `tfvars`, computed locals, `foreach`-driven subnets, dynamic security-rule blocks, lifecycle guards, and three techniques I used for changing a configuration without destroying live infrastructure: `terraform state mv`, the declarative moved block, and `terraform import`.

Three findings stand out:

- **Tier isolation is enforced, not assumed.** I confirmed that security-list rules on the private tier never source from outside the virtual cloud network.
- **The request path works end to end.** I verified that the load balancer's listener, backend set, and health checker route a real request successfully.
- **State changes are safe.** I demonstrated both state-refactoring techniques and the import of a hand-created resource live, with **zero unintended creates or destroys** in `terraform plan`.

I fully implemented the intended **three-path defense model**: public load-balanced traffic, bastion-mediated administrative access, and outbound-only NAT egress, with no other route into the private subnet. Three limitations qualify that conclusion:

- I left the Bastion service's client CIDR allow list at its default, `0.0.0.0/0`, for lab convenience;
- I stored Terraform state *locally*, with no locking; and

- I have not exercised the build under concurrent multi-operator use or real traffic load.

Before I reuse this pattern for a team or a production workload, I would make three changes first: **move the state backend to OCI Object Storage**, **narrow the Bastion allow list** to known operator addresses, and **add a genuine database tier** behind the application layer.

Contents

List of Abbreviations	vi
List of Figures	viii
List of Tables	x
1 Introduction and Scope	1
1.1 Background and Motivation	1
1.2 Objective	2
1.3 Scope and Boundaries	2
1.4 Key Stakeholders	3
1.5 Main Risks and Uncertainties	3
1.6 How This Documentation Is Organized	4
2 Architecture Concepts and Trade-offs	5
2.1 The General Tiered Application Model	5
2.2 Load Balancer Concepts	6
2.3 Security and Operational Constraints Shaping the Design	7
2.4 Evidence and Verification Basis	7
2.5 Design Trade-offs Considered	8
3 Terraform Implementation	10
3.1 Manual Console Build Before Automation	10
3.1.1 Finding: A Missing IAM Policy Blocked the Bastion Service	12
3.1.2 Finding: Cloud Shell's FIPS-Restricted Client Rejected My Ed25519 Key	13
3.1.3 File Storage, Built the Same Way	15
3.2 Implementation Method: Variables and tfvars	16
3.3 Configuration Framework: Locals and Repetition	19
3.4 Change-Safety Criteria: Lifecycle Rules and State Refactoring	22
3.5 State as the Source of Truth	24

3.6	Assumptions and Limitations	26
3.7	Environment Teardown	26
4	Deployment Results and Verification	28
4.1	The Deployed Network Enforces Strict Tier Isolation	28
4.2	The Load Balancer Health-Checks and Routes Traffic End-to-End, Verified Live	31
4.3	Private Instance Access Is Ephemeral, Audited, and Requires No Standing At- tack Surface	35
4.4	State Refactoring and Import, Demonstrated Live	36
4.5	Risk Analysis	38
4.6	Key Unknowns	39
5	Conclusions	40
5.1	Main Conclusion	40
5.2	Strongest Supporting Evidence	40
5.3	Biggest Uncertainty	41
5.4	Strategic Implications and Recommendation	41
5.5	Confidence Level and Final Judgement	41
6	Recommendations and Next Steps	42
6.1	Further Testing Needed	42
6.2	Data to Validate	42
6.3	Recommended Actions	42
6.4	Monitoring Indicators	43
6.5	Open Questions	43
	Appendix	44
.1	Glossary	45
.2	Command Reference	46
	References	47

List of Abbreviations

OCI	Oracle Cloud Infrastructure
VCN	Virtual Cloud Network
CIDR	Classless Inter-Domain Routing
IGW	Internet Gateway
NAT	Network Address Translation
NSG	Network Security Group
NFS	Network File System
LB	Load Balancer
IaC	Infrastructure as Code
HCL	HashiCorp Configuration Language
OCID	Oracle Cloud Identifier
AD	Availability Domain
TTL	Time to Live
SSH	Secure Shell
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
API	Application Programming Interface
CLI	Command-Line Interface
JSON	JavaScript Object Notation

DNS	Domain Name System
URL	Uniform Resource Locator
VM	Virtual Machine
IP	Internet Protocol

List of Figures

3.1	OCI Console: <code>explore-vcn</code> , CIDR <code>10.1.0.0/16</code> , compartment <code>intern-02-ali-youssef-cmp</code>	11
3.2	<code>explore-public-subnet</code> (<code>10.1.1.0/24</code>) and <code>explore-private-subnet</code> (<code>10.1.2.0/24</code>), both Available, matching the two-tier split <code>main.tf</code> enforces through <code>prohibit_public_ip_on_vnic</code>	11
3.3	<code>explore-linux</code> , launched in <code>AD-1</code> , compartment <code>intern-02-ali-youssef-cmp</code>	12
3.4	<code>explore-linux</code> 's Networking tab: Public IPv4 address is blank, Private IPv4 address is <code>10.1.2.148</code> , in <code>explore-private-subnet</code>	12
3.5	The Create Bastion form, fully and correctly filled in: target VCN <code>explore-vcn</code> , target subnet <code>explore-private-subnet</code> . Submitting this silently failed with the policy gap described above.	13
3.6	Cloud Shell's Network Configuration panel, before creating a private network definition into <code>explore-vcn</code>	14
3.7	<code>explore-cloudshell-net</code> , created and attached, giving Cloud Shell a VNIC inside my VCN.	14
3.8	Cloud Shell terminal mid-session, working through the SSH connection to <code>explore-linux</code> at <code>10.1.2.148</code>	15
3.9	<code>explore-fs</code> , Active, in <code>AD-1</code>	15
3.10	Creating the export at path <code>/explore-export</code> , through <code>explore-mount-target</code>	16
3.11	The completed export: <code>/explore-export</code> , Active, correctly linked to <code>explore-fs</code> through <code>explore-mount-target</code>	16
3.12	One dynamic block, driven by a six-element list, expands into six nested <code>ingress_security_rules</code> blocks, each choosing <code>tcp_options</code> or <code>udp_options</code> by its own protocol value.	22
3.13	Both paths prevent the same spurious destroy-and-recreate; they differ only in who is protected by the fix. Solid outlines mark both paths as implemented and evaluated options, not a specified-versus-implemented distinction.	24

3.14	Simplified shape of <code>terraform.tfstate</code> . The <code>index_key</code> field is how <code>for_each</code> addresses (Section 3.3) appear in state, matching each subnet's own map key.	25
3.15	Local versus remote (Object Storage) state backend. The dashed node marks the remote backend as specified in <code>provider.tf</code> but not currently enabled; the solid node marks the local backend as the implemented, evaluated configuration.	26
4.1	Full Week 2 network topology, enclosed in VCN <code>tf-lab-vcn-AliYousef</code> (10.1.0.0/16). All names, IPs, and CIDR blocks are taken from live Terraform state; the NAT and Bastion paths are shown as outbound-only or session-scoped rather than permanently open.	29
4.2	OCI Console: <code>tf-lab-vcn-AliYousef</code> , CIDR 10.1.0.0/16, compartment <code>intern-02-ali-youssef-cmp</code> , region <code>me-jeddah-1</code>	31
4.3	Request path for a single HTTP request, and the parallel, continuous health-check loop that decides which backends are eligible to receive it. All steps are implemented and verified against the live environment.	33
4.4	The load balancer, Active, with its public IP 141.147.131.235.	33
4.5	Listener <code>tf-lab-listener-AliYousef</code> : HTTP on port 80, forwarding to backend set <code>tf-lab-backend-set-AliYousef</code>	34
4.6	Backend set with its round-robin policy and one registered backend.	34
4.7	The individual backend entry: private IP and application port, captured mid-deployment while the health check was still converging.	35
4.8	The manually created NSG (<code>tf-lab-imported-nsg-AliYousef</code>) in the OCI Console, prior to being brought under Terraform management with the command above.	38

List of Tables

1.1	Stakeholders in the Week 2 build and this documentation.	3
2.1	The general four-tier application architecture model.	6
2.2	The three components of an OCI load balancer.	7
2.3	Evidence types used to support claims in this documentation.	8
2.4	Comparison of the four options for reaching a private OCI instance.	9
3.1	Every variable declared in <code>weekTwo/variables.tf</code>	18
3.2	<code>count</code> vs. <code>for_each</code>	20
3.3	Criteria for choosing a change-safety mechanism.	23
4.1	CIDR allocation for the deployed VCN.	28
4.2	All ingress rules deployed by <code>weekTwo</code> . Both lists also egress <code>0.0.0.0/0</code> on all protocols.	30
4.3	Route tables. The private route still points <code>0.0.0.0/0</code> at the NAT gateway, giving outbound-only internet access without ever accepting an inbound connection, because NAT gateways are stateful and egress-only by design.	30
4.4	Risks identified in the current configuration.	38
6.1	Indicators worth monitoring if this environment moves beyond a single-operator lab.	43
2	Terms used throughout this documentation.	45
3	Terraform commands referenced in this documentation, all run against <code>weekTwo/</code> during my work on this lab.	46

Chapter 1

Introduction and Scope

In Week 1 of this internship, I built a single-tier environment on Oracle Cloud Infrastructure (OCI): one Virtual Cloud Network (VCN), one public subnet, and one compute instance reachable directly from the internet. That is *not* how production workloads are deployed: a database, and often the application server itself, should never be reachable from the public internet directly. Week 2's assignment, *Enterprise OCI Network Design*, asked for the opposite: a network with public and private tiers, a load balancer fronting the private tier, and a managed, audited way to reach the private tier for operations work. This documentation is my combined deliverable for that assignment. I cover the OCI concepts and architecture behind my build and the Terraform mechanics I used to implement it, as one cross-referenced write-up rather than two separate ones.

1.1 Background and Motivation

Splitting an application into public and private tiers is a **defense-in-depth** strategy: it limits what an attacker can reach even if one layer is compromised, and it lets each layer scale and be secured independently. Building that split by hand, resource by resource, does not scale past a first lab exercise. I needed the network definition itself to be **reusable** across compartments and owners, **safely refactorable** without destroying live infrastructure, and **auditable** from a diff rather than from console clicks. I used Terraform to satisfy that second requirement, and applied it to the OCI networking model to satisfy the first. Neither half is complete without the other. The architecture chapter explains *what* I built and why the placement of every resource matters, and the approach and findings chapters explain *how* I built and verified it.

1.2 Objective

The primary question I set out to answer is: *does the network I deployed actually enforce the three-tier defense model it was designed around, and is the Terraform configuration I built it with safe to change without destroying resources that do not need to change?* Two secondary questions follow from it:

- which of the practical options for reaching a private OCI instance best satisfies security, cost, and operational-overhead constraints; and
- which Terraform state-management technique, an imperative `state mv` command or a declarative `moved` block, is the better default for a team rather than a single operator.

1.3 Scope and Boundaries

I scoped this build to a **single OCI region**, `me-jeddah-1`, and a **single compartment**, `intern-02-ali-youssef-cmp`, within a single availability domain. The assignment scope collapses the classic web and application tiers into one private “application” tier sitting behind one load balancer, and I did not build a separate database tier: the application’s data is static files served from OCI File Storage rather than a relational database. That is a *deliberate simplification* for a two-week lab, not a shortcut I took by accident. The same public/private split, the same NAT-for-egress-only pattern, and the same rule that nothing reaches the private subnet except through the load balancer or the bastion apply exactly as they would in a four-tier design; I just had fewer tiers to draw. I followed the `tf-lab-*-AliYousef` naming convention I established in Week 1 throughout `weekTwo/`. I did *not* cover multi-region or multi-availability-domain high availability, load testing under real traffic, or a production compliance review in this build. I name those explicitly as follow-up work in Chapter 6.

1.4 Key Stakeholders

Table 1.1: Stakeholders in the Week 2 build and this documentation.

Stakeholder	Interest or Role	Relevance
Operator (Ali Yousef)	Designs, deploys, and maintains the environment via Terraform	Author of both the configuration and this documentation; the primary reader of the approach and findings chapters
Ejada Cloud Build Track	Reviews the assignment deliverable against the stated learning objectives	Target audience for this documentation; cares most about correctness of the architecture and clarity of the Terraform explanation
Application end users	Send HTTP requests to the load balancer's public IP	Affected by the correctness of the listener, backend set, and health-check configuration described in Chapter 4
Future maintainers	Would extend this configuration, for example by adding a database tier or enabling the remote backend	Rely on the state-management practices in Chapter 3 to change the configuration safely

1.5 Main Risks and Uncertainties

Three issues are worth flagging before the detailed walkthrough in Chapter 4:

- I left the Bastion service's client CIDR allow list at its default, `0.0.0.0/0`: **fine for a short-lived training environment, wrong for anything longer-lived.**
- I stored state only on my own local disk, with **no locking and no shared source of truth**, which becomes a real constraint the moment a second person needs to run Terraform against the same infrastructure.
- I verified the environment functionally correct end to end (a live `curl` against the load balancer's public IP returned the application's HTML), but I have *not* exercised it under concurrent access, sustained load, or a failure of the single availability domain it lives in.

1.6 How This Documentation Is Organized

In Chapter 2, I set out the conceptual context: the general tiered application model, load balancer concepts, and the trade-offs between the practical options for reaching a private instance. In Chapter 3, I explain the Terraform mechanics I used to implement that architecture: variables, locals, `for_each`, dynamic blocks, lifecycle rules, and the state file as the source of truth. Chapter 4 walks through the network as I actually deployed and verified it, including console evidence and live demonstrations of safe state refactoring and resource import. Chapter 5 turns the evidence into my final judgement, and Chapter 6 lists the concrete next actions and open questions I would tackle next. The appendix collects a glossary of terms and a command reference.

Chapter 2

Architecture Concepts and Trade-offs

In this chapter, I set out the conceptual background I relied on before building anything: the general model an enterprise network design follows, the roles the load balancer's three components play, the trade-offs between the practical options for reaching a private instance, and the basis on which I ask you to trust every claim in this documentation.

2.1 The General Tiered Application Model

Splitting an application into tiers is a defense-in-depth strategy: it limits what an attacker can reach even if one layer is compromised, and it lets each layer scale and be secured independently. The classic model used in enterprise OCI designs has four tiers, summarized in Table 2.1.

Table 2.1: The general four-tier application architecture model.

Tier	Typical components	Subnet placement	Internet exposure
Web / Presentation	Load balancer, reverse proxy, static content edge	Public	Yes: the only tier the internet should ever touch directly
Application	Business logic, API servers, backend processes	Private	No: reachable only from the web tier’s subnet, plus a bastion for admin access
Database	Relational/NoSQL database instances	Private, often a dedicated, even more restricted subnet	No: reachable only from the application tier, never from the web tier or the internet
Storage	Block Volumes, Object Storage, File Storage (NFS)	Private (File Storage mount targets live on a private subnet)	No: reachable only from the tier(s) that mount or access it

One rule falls out of this table, and it is worth stating explicitly because it is the single design decision that drives every security list and route table described in Chapter 4: **a resource should sit in the most restrictive subnet that still lets it do its job.** A load balancer must be reachable from the internet, so it goes in a public subnet. An application server only ever needs to be reached by the load balancer, not by a browser directly, so it belongs in a private subnet with no public IP at all, even though it still needs outbound internet access for OS package updates via a NAT gateway.

For this build, I collapsed the web and application tiers into a single private “application” tier sitting behind one load balancer, and I did not build a separate database tier: the application’s data is static files served from OCI File Storage rather than a database. That is a deliberate simplification I made, not a shortcut. I left address space unused on purpose in the compartment’s subnet design, so I can add a database subnet in a later phase without renumbering anything that already exists.

2.2 Load Balancer Concepts

The OCI Load Balancing service sits in the public subnet and is the only thing on the internet-facing side of this architecture with a public IP. It terminates client connections and forwards them to backends it has already health-checked, so a client can never reach an unhealthy or nonexistent backend directly, and the backend itself never needs a public IP at all. A load balancer is composed of three objects that always appear together, summarized in Table 2.2.

Table 2.2: The three components of an OCI load balancer.

Component	Responsibility
Listener	What the client connects to: a protocol (HTTP/HTTPS/TCP) and port on the load balancer's public IP. It does not know or care what backends exist; it just hands every accepted connection to a named backend set.
Backend set	A named pool of backend IP:port pairs plus a load-balancing policy (round robin, least connections, or IP hash) and a health checker. This is the layer that decides which backend a given request actually goes to.
Health checker	Continuously probes every backend in the set (protocol, port, URL path, expected return code, interval, timeout, retry count) and marks each one OK or Critical. The backend set only ever routes traffic to backends currently marked OK.

OCI load balancers come in two shapes: a fixed shape (10 Mbps, 100 Mbps, 400 Mbps, 8000 Mbps) or a flexible shape with a configurable bandwidth range. I cover which shape and bandwidth range I actually used, and how I configured the three components above in Terraform, as a finding in Section 4.2.

2.3 Security and Operational Constraints Shaping the Design

Two operational constraints shape the design beyond the tier model itself.

- **NAT gateways are egress-only by design.** A NAT gateway is stateful and egress-only: it gives a private subnet outbound internet access without ever accepting an inbound connection, which is what lets the private application tier fetch OS updates without becoming reachable from the internet.
- **A TCP-only health check can be misleading.** An application process can be alive while its web server is not actually serving content. An HTTP-level check with an expected status code catches that failure mode, at the cost of needing enough retries and interval time to tolerate normal startup delay, for example a cloud-init script still mounting a network file system before the application service can start.

I revisit both constraints directly in the deployed configuration I examine in Chapter 4.

2.4 Evidence and Verification Basis

I did not build this documentation from external literature. It covers one specific environment I deployed myself, so every claim needs a different kind of grounding: reproducible evidence

from the environment itself rather than a citation. Table 2.3 summarizes the three kinds of evidence I use throughout this documentation, each referenced by figure or table number at the point it supports a claim.

Table 2.3: Evidence types used to support claims in this documentation.

Evidence type	What it confirms	Basis for trust
Terraform state (<code>terraform. tfstate</code> , via <code>state show</code>)	Exact attribute values OCI assigned to each resource (IPs, OCIDs, CIDR blocks)	Written by OCI’s own API responses at apply time, not hand-typed
OCI Console screenshots	That a resource exists, is active, and is configured as claimed	Captured directly from the live environment during this lab
Live command output (<code>curl</code> , <code>terraform plan</code>)	That the deployed system behaves as designed, not just that it exists	Executed against the real endpoint or the real state file at the time of writing

2.5 Design Trade-offs Considered

I gave the compute instance in this build no public IP, by design. That is correct from a security standpoint, but it raises an operational question: how do I reach it to troubleshoot, patch, or inspect it? Four approaches are commonly compared for OCI. Table 2.4 evaluates all four on the same criteria before I explain, in Chapter 4, which one I used and why.

Table 2.4: Comparison of the four options for reaching a private OCI instance.

Option	How it works	Pros	Cons
OCI Cloud Shell	Browser-based shell running inside OCI's network, with network access to VCN resources if configured	No client install; runs entirely inside OCI's trust boundary; free	Only reaches resources reachable from its own network path; not a general-purpose bastion without extra networking setup; session-based, not persistent
Jump VM (manual bastion host)	A small compute instance with a public IP in the public subnet, SSH'd into first and then SSH'd again from, to reach the private instance	Full control; works with any protocol, not just SSH; no managed-service dependency	An entire extra VM to patch, monitor, and secure; a permanent open SSH port as attack surface; no built-in session audit trail
OCI Bastion Service	A managed OCI service that brokers short-lived SSH sessions to a target resource in a specific subnet; no public IP or open inbound port is ever created on the target	Fully managed; every session is logged and time-bounded; the target's security list only needs to allow SSH from inside the VCN; no standing attack surface when idle	Adds console/CLI overhead per session; sessions expire and must be recreated; still requires an SSH key pair
SSH port forwarding / tunneling	Using an existing reachable host to forward a local port to the private instance's SSH port, then connecting to <code>localhost</code>	Works with any SSH client and many GUI tools; can be layered on top of any of the above, including Bastion	A technique, not a service by itself; still depends on a reachable host to tunnel through; offers no security or audit properties of its own

One pattern holds across all four rows: the more managed an option is, the less standing infrastructure it leaves for the operator to secure, at the cost of some per-session friction. That trade-off decided the choice I made for this build, which I detail in Section 4.3.

Chapter 3

Terraform Implementation

In Week 1, I used Terraform at the level of resources, variables, and outputs: enough to stand up a fixed environment. Week 2's assignment asked for more: a configuration that is

- **reusable** across compartments and owners,
- **repeatable** without copy-paste,
- **protected** against careless changes to specific resources, and
- **safely refactorable** without destroying real infrastructure.

In this chapter, I explain each mechanism I used to meet those requirements; Chapter 4 then shows me exercising each one live against the real environment. Every code listing below is copied verbatim from my files in `weekTwo/`, not simplified or rewritten for this documentation.

3.1 Manual Console Build Before Automation

Before writing a single resource block for Week 2, I built the same network by hand in the OCI Console once, under an `explore-` naming prefix so nothing here is confused with the Terraform-managed `tf-lab-*-AliYousef` resources documented in Chapter 4. The point of doing this manually first was to understand what each argument in `main.tf` actually controls before letting Terraform set it for me, particularly for the load balancer, which is the resource with the most interdependent configuration in the whole design.

I built the VCN, both subnets, both gateways, both route tables, and both security lists by hand, typing out the same six ingress rules per security list that `oci_core_security_list` in `main.tf` generates from a `dynamic` block. Figure 3.1 and Figure 3.2 confirm the VCN and its two subnets came up with the CIDR blocks and public/private access settings I intended.

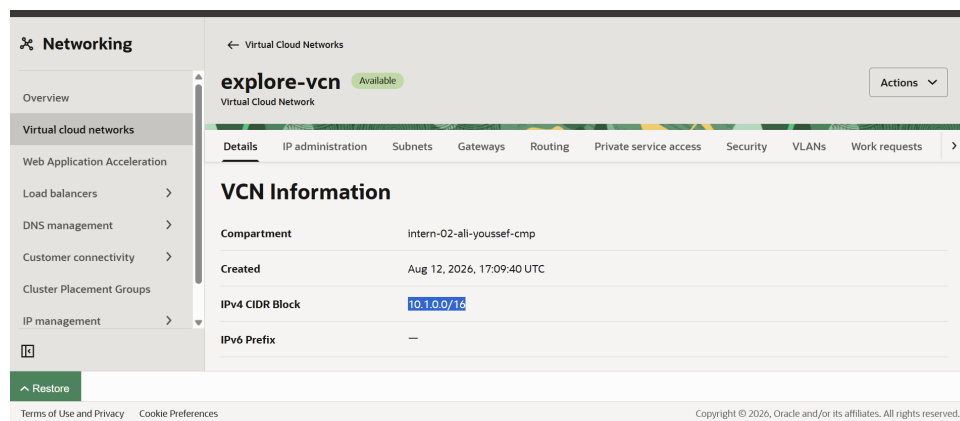


Figure 3.1: OCI Console: `explore-vcn`, CIDR `10.1.0.0/16`, compartment `intern-02-ali-youssef-cmp`.

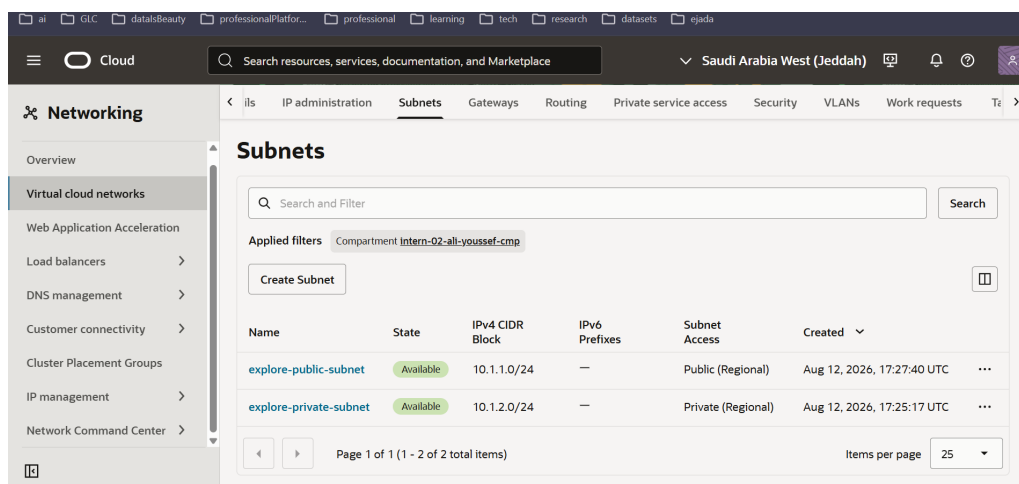


Figure 3.2: `explore-public-subnet` (`10.1.1.0/24`) and `explore-private-subnet` (`10.1.2.0/24`), both Available, matching the two-tier split `main.tf` enforces through `prohibit_public_ip_on_vnic`.

I then launched `explore-linux` into the private subnet with no public IP, and confirmed from the instance's own Networking tab that it received only a private address, with no path in from the internet, shown in Figure 3.3 and Figure 3.4.

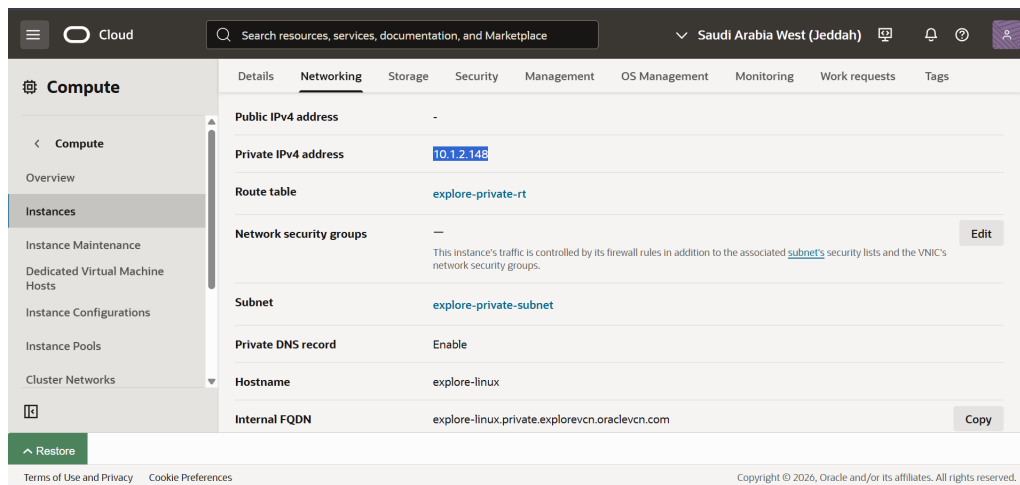


Figure 3.3: explore-linux, launched in AD-1, compartment intern-02-ali-youssef-cmp.

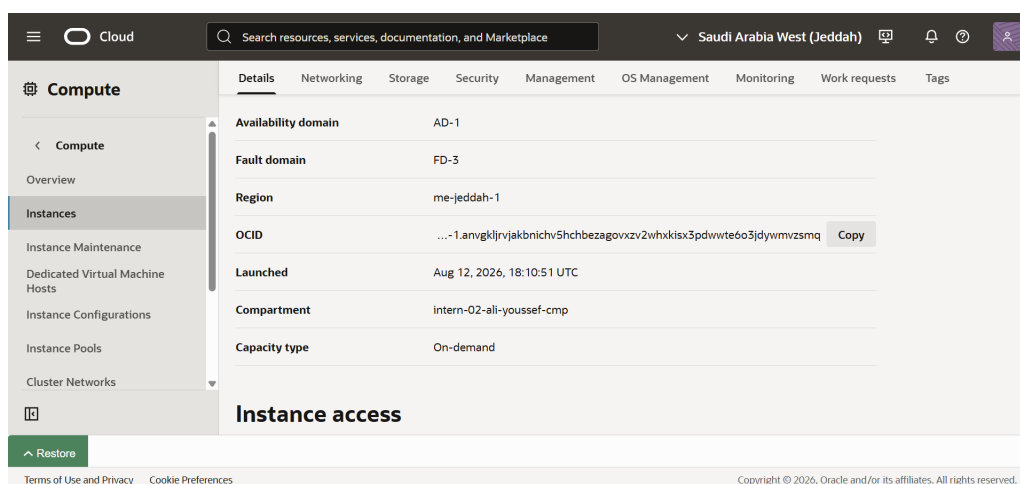


Figure 3.4: explore-linux's Networking tab: Public IPv4 address is blank, Private IPv4 address is 10.1.2.148, in explore-private-subnet.

3.1.1 Finding: A Missing IAM Policy Blocked the Bastion Service

Reaching a private instance with no public IP is exactly the problem the Bastion service solves, so the next step was creating one against explore-private-subnet (Figure 3.5). Submitting the form did nothing visibly, so I opened the browser's network inspector and found the actual request:

Listing 3.1: The Bastion creation request, captured from DevTools

```
1 POST bastion.me-jeddah-1.oc1.oraclecloud.com/20210331/bastions -> 404
2 { "code": "NotAuthorizedOrNotFound",
```

```
3 "message": "Authorization failed or requested resource not found."}
```

OCI deliberately returns 404 rather than 403 for authorization failures, so a client cannot distinguish "this does not exist" from "you cannot see this." I confirmed it was the latter: the Identity & Security → Policies page in this compartment returned the identical error when I tried to simply list existing policies, which means my account cannot even read IAM policy state here, let alone one covering `bastion-family`. This is an environment gap, not a configuration mistake on my part, and it would need a tenancy administrator to add a policy of the form `Allow group <my-group> to manage bastion-family in compartment intern-02-ali-youssef-cmp` before the Bastion service becomes usable in this compartment.

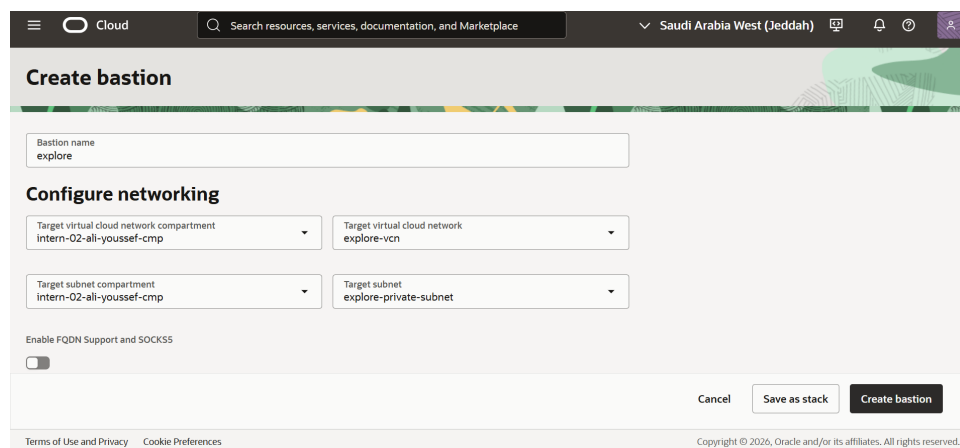


Figure 3.5: The Create Bastion form, fully and correctly filled in: target VCN `explore-vcn`, target subnet `explore-private-subnet`. Submitting this silently failed with the policy gap described above.

3.1.2 Finding: Cloud Shell's FIPS-Restricted Client Rejected My Ed25519 Key

With Bastion unavailable, I used OCI Cloud Shell as an alternative path in, attaching it directly to `explore-private-subnet` through a private network definition so it would receive its own address inside my VCN rather than needing any public-facing hop (Figure 3.6, Figure 3.7).

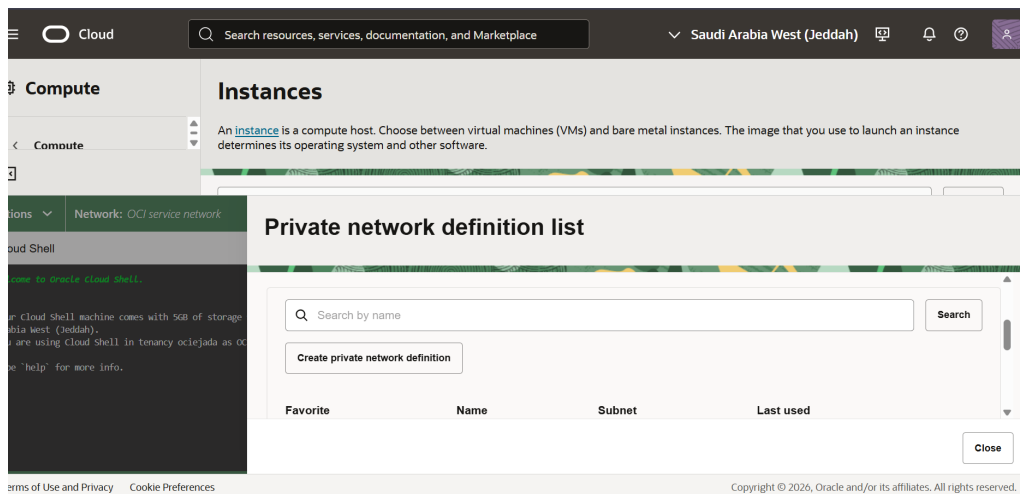


Figure 3.6: Cloud Shell’s Network Configuration panel, before creating a private network definition into `explore-vcn`.

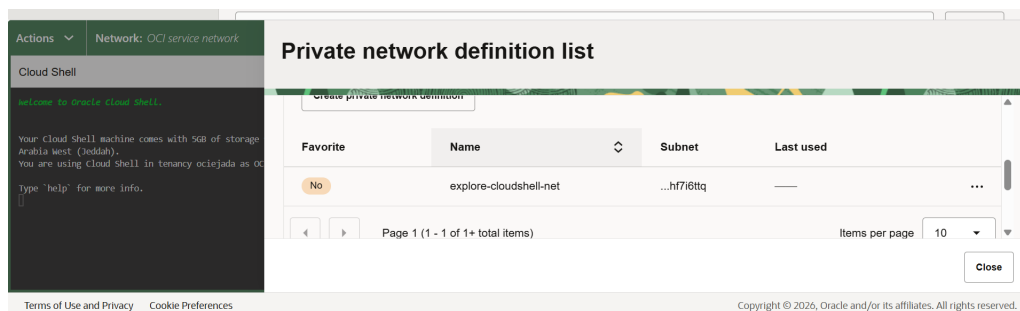


Figure 3.7: `explore-cloudshell-net`, created and attached, giving Cloud Shell a VNIC inside my VCN.

Connectivity itself worked once I corrected a copy-paste error in my own security list, where five of six ingress rules on `explore-private-sl` had inherited the wrong source CIDR (`10.1.1.0/24` instead of `10.1.0.0/16`) from the rule above them, a mistake worth noting precisely because it is the same class of error the `dynamic` block in Section ?? exists to prevent. After fixing it, SSH reached the instance but authentication still failed:

Listing 3.2: SSH failing against an Ed25519 key from Cloud Shell

```
1 $ ssh -i oci_vm opc@10.1.2.148
2 sign_and_send_pubkey: no mutual signature supported
3 opc@10.1.2.148: Permission denied (publickey,gssapi-keyex,gssapi-with-mic
4 )
5 $ ssh -V
6 OpenSSH_8.0p1, OpenSSL 1.1.1k FIPS 25 Mar 2021
```

Cloud Shell’s bundled OpenSSH client is built against a FIPS-restricted OpenSSL, and FIPS mode does not permit Ed25519 signatures, since Curve25519 is not on the FIPS-approved curve list. My original key pair is Ed25519, so the client could hold the key but was cryptographically unable to sign with it, independent of whether the key was authorized on the server. The working fix was generating a fresh RSA key pair (RSA is FIPS-approved) directly inside Cloud Shell and delivering its public half to `explore-linux` through OCI’s Run Command instance-agent plugin rather than SSH, since SSH was the very thing that was failing (Figure 3.8).

```
Cloud Shell
aliyousef@cloudshell:~ (me-jeddah-1)$ chmod 600 oci_vm
aliyousef@cloudshell:~ (me-jeddah-1)$ ssh -i oci_vm opc@explore-linux_private_ip
bash: syntax error near unexpected token `newline'
aliyousef@cloudshell:~ (me-jeddah-1)$ ssh -i oci_vm opc@10.1.2.148
```

Figure 3.8: Cloud Shell terminal mid-session, working through the SSH connection to `explore-linux` at `10.1.2.148`.

3.1.3 File Storage, Built the Same Way

I built the File Storage stack as three separate resources rather than through the bundled wizard, to see the file system, mount target, and export as distinct pieces the way `main.tf` treats them: `oci_file_storage_file_system`, `oci_file_storage_mount_target`, and `oci_file_storage_export` are three separate resource blocks, not one.

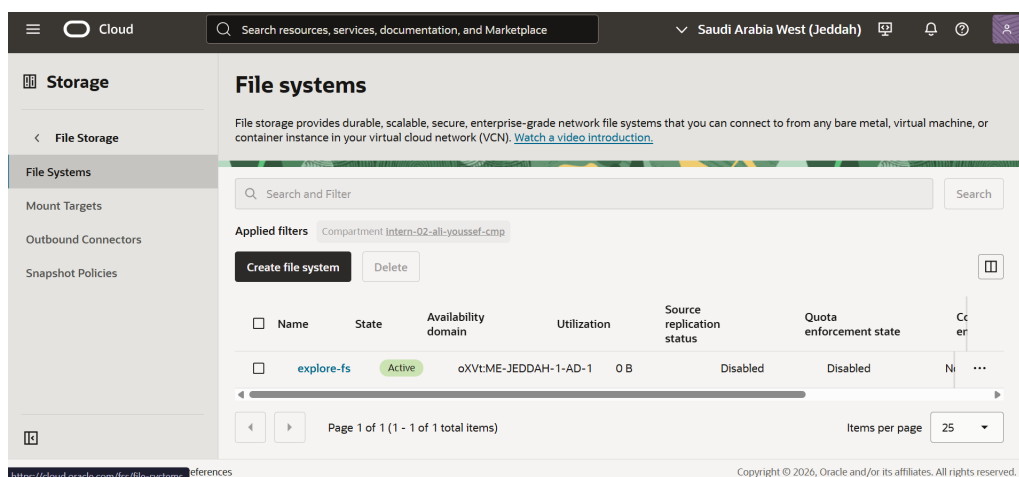


Figure 3.9: `explore-fs`, Active, in AD-1.

Create export

Create a new export and file system in the mount target named "explore-mount-target". [Learn more](#)

Export information

Exports control which file systems are available to a given mount target. Create a new export to make your file system available through the selected mount target. [Learn more](#)

Export path
/explore-export

Use secure export options
☒

When selected, export options are applied to NFS clients requiring them to use privileged source ports. After creating exports, you can edit the NFS client export options to adjust security for any export. See [Working with NFS Export options](#) for more information.

Use LDAP for group list
☐

Cancel Save as stack Create

Terms of Use and Privacy Cookie Preferences Copyright © 2026, Oracle and/or its affiliates. All rights reserved.

Figure 3.10: Creating the export at path /explore-export, through explore-mount-target.

Storage

File Storage

File Systems

Mount Targets

Outbound Connectors

Snapshot Policies

File system details

/explore-export Active

Export

Actions Mount commands

Details NFS client export options

General information

OCID	...e_jeddah_1.aaaaa4np2yfgxernjswillqojxwiotnmuvwuzlemrqqljrfvqwlljr	Copy
Created	Wed, Aug 12, 2026, 18:42:34 UTC	
File system	explore-fs	
Mount target	explore-mount-target	
Use LDAP for group list	Disabled	Edit

Terms of Use and Privacy Cookie Preferences Copyright © 2026, Oracle and/or its affiliates. All rights reserved.

Figure 3.11: The completed export: /explore-export, Active, correctly linked to explore-fs through explore-mount-target.

Doing this by hand once, including hitting the two real environment issues above, is what turns the Terraform configuration in the rest of this chapter from code I copied into code I can explain: every resource block below corresponds to a specific screen, a specific set of required arguments, and in two cases, a specific failure mode I had already debugged before automating it away.

3.2 Implementation Method: Variables and tfvars

A Terraform variable is an input: a named, typed placeholder that resource blocks reference instead of a literal value. The point is that the resource files never contain a tenancy OCID, a

compartment OCID, or a CIDR block written directly. They contain `var.tenancy_ocid`, `local.my_compartment_id`, `var.subnet_tiers`, and so on. That indirection is what makes the exact same configuration re-runnable for a different person, compartment, or region without touching a single resource block.

Listing 3.3: A representative slice of `weekTwo/variables.tf`

```
1 variable "tenancy_ocid" {
2   description = "OCI tenancy OCID"
3   type       = string
4   sensitive   = true
5 }
6
7 variable "region" {
8   description = "OCI region"
9   type       = string
10  default     = "me-jeddah-1"
11 }
```

I supply values from `terraform.tfvars`, a file Terraform loads automatically if present. I listed this file in `.gitignore` and **never committed** it, because it holds actual credentials: the tenancy OCID, user OCID, API key fingerprint, and private key path are all marked `sensitive = true` in `variables.tf`, so Terraform also redacts them from console output and plan diffs. Table 3.1 lists every variable I declared in the configuration.

Table 3.1: Every variable declared in `weekTwo/variables.tf`.

Variable	Type	Purpose
<code>tenancy_ocid</code>	string, sensitive	OCI tenancy OCID (required, no default)
<code>user_ocid</code>	string, sensitive	OCI user OCID (required, no default)
<code>fingerprint</code>	string, sensitive	API key fingerprint (required, no default)
<code>private_key_path</code>	string, sensitive	Path to the API private key (required, no default)
<code>region</code>	string	OCI region, default <code>me-jeddah-1</code>
<code>compartment_ocid</code>	string	Target compartment OCID (required, no default)
<code>instance_shape</code>	string	Compute shape, default <code>VM.Standard.E2.1.Micro</code>
<code>ssh_public_key_path</code>	string	Local path to the SSH public key
<code>subnet_tiers</code>	map(object)	One entry per subnet tier; drives the <code>for_each</code> in Section 3.3
<code>public_ingress_rules</code>	list(object)	Ingress rules for the public security list; drives the dynamic block in Section 3.3
<code>private_ingress_rules</code>	list(object)	Same, for the private security list
<code>app_port</code>	number	Port the application listens on, default 8080
<code>lb_min_bandwidth_mbps</code>	number	Flexible load balancer minimum bandwidth, default 10
<code>lb_max_bandwidth_mbps</code>	number	Flexible load balancer maximum bandwidth, default 10
<code>bastion_client_cidr_allow_list</code>	list(string)	CIDRs allowed to open a Bastion session, default <code>0.0.0.0/0</code>
<code>bastion_max_session_ttl_in_seconds</code>	number	Max Bastion session lifetime, default 10800

I typed two variables as structured objects rather than scalars: `subnet_tiers` is a map of objects consumed by `for_each`, and the ingress-rule variables are lists of objects consumed by a dynamic block. Typing them precisely, rather than a loose `map(any)`, means **Terraform validates the shape of `terraform.tfvars` before planning**, catching a typo'd key or wrong type at `terraform validate` time instead of as a confusing apply-time error.

3.3 Configuration Framework: Locals and Repetition

Local values

A *local* is the opposite of a variable: instead of an external input, it is a value computed once, inside the configuration, from other values already available. Locals exist purely to **avoid repeating an expression**.

Listing 3.4: The complete weekTwo/locals.tf

```
1 locals {
2   my_compartment_id = data.oci_identity_compartments.mine.compartments
      [0].id
3
4   name_prefix = "tf-lab"
5   owner       = "AliYousef"
6
7   common_freeform_tags = {
8     project      = "ejada-oci-terraform-lab"
9     environment  = "week2-lab"
10    owner        = local.owner
11    managed_by   = "terraform"
12  }
13 }
```

I resolve `my_compartment_id` once from a data-source lookup, then reference it from every resource as `local.my_compartment_id` instead of repeating the same lookup expression each time. `name_prefix` and `owner` build every `display_name` in the project, so re-running the configuration under a different owner is a one-line edit for me, not a find-and-replace across nine files. `common_freeform_tags` guarantees the same four tag keys on every taggable resource instead of risking one of four hand-typed copies drifting. The distinction I keep straight is that a **variable** is something Terraform cannot compute for itself, while a **local** is something it can, and the only reason to name it is to avoid repeating the computation.

count versus for_each

Both `count` and `for_each` create multiple instances of a resource from one block instead of copy-pasting it. They differ in how each instance is addressed in state, which matters when the input list changes later, summarized in Table 3.2.

Table 3.2: count vs. for_each.

	count	for_each
Input	A number, or the length of a list	A map or a set of strings
Instance address	<code>resource.name[0]</code> , <code>[1]</code> , ... (numeric index)	<code>resource.name["key"]</code> (map key)
Item removed from the middle of the input	Every later instance shifts index, so Terraform may destroy/recreate resources that did not actually change	Only the removed key is destroyed; every other instance's address is untouched
Best for	A fixed number of identical resources where order never matters	Individually meaningful resources that may be added or removed over time

Listing 3.5: weekTwo/main.tf – subnets built with for_each

```

1 resource "oci_core_subnet" "tier" {
2   for_each = var.subnet_tiers
3
4   compartment_id      = local.my_compartment_id
5   vcn_id              = oci_core_vcn.terraform_vcn.id
6   display_name        = "${local.name_prefix}-${each.key}-subnet-${
7     {local.owner}}"
8   cidr_block          = each.value.cidr_block
9   dns_label           = each.value.dns_label
10  route_table_id      = each.value.is_public ?
11    oci_core_route_table.public_route_table.id : oci_core_route_table.
12    private_route_table.id
13  security_list_ids    = each.value.is_public ? [
14    oci_core_security_list.web_tier_security_list.id] : [
15    oci_core_security_list.private_tier_security_list.id]
16  prohibit_public_ip_on_vnic = !each.value.is_public
17 }

```

With `for_each`, this one block creates exactly two subnet resources, which I address in state as `oci_core_subnet.tier["public"]` and `oci_core_subnet.tier["private"]`. **If I add a third tier to `var.subnet_tiers` later, Terraform would plan to create one new subnet and leave the existing two completely alone.**

Dynamic blocks

Some Terraform arguments are not simple key-value pairs but nested blocks, such as `ingress_security_rules { ... }` inside a security list. A dynamic block generates that repeated nested block from a list or map at plan time, the same way `for_each` generates repeated top-level resources.

Listing 3.6: weekTwo/main.tf – the dynamic ingress block, with a nested dynamic block inside it

```
1 dynamic "ingress_security_rules" {
2   for_each = var.public_ingress_rules
3   content {
4     source      = ingress_security_rules.value.source
5     source_type = "CIDR_BLOCK"
6     protocol    = ingress_security_rules.value.protocol
7     stateless   = false
8     description = ingress_security_rules.value.description
9
10    dynamic "tcp_options" {
11      for_each = ingress_security_rules.value.protocol == "6" ? [1] : []
12      content {
13        min = ingress_security_rules.value.port_min
14        max = ingress_security_rules.value.port_max
15      }
16    }
17
18    dynamic "udp_options" {
19      for_each = ingress_security_rules.value.protocol == "17" ? [1] : []
20      content {
21        min = ingress_security_rules.value.port_min
22        max = ingress_security_rules.value.port_max
23      }
24    }
25  }
26 }
```

Figure 3.12 shows what this expands into for my private security list, whose input is a six-element list: one `content { }` block is generated per list element, and within each of those, the two inner dynamic blocks each generate zero or one nested block depending on whether that rule's protocol is TCP or UDP. *Without* this, adding a seventh ingress rule would mean hand-writing another nested block in `main.tf`. *With* it, adding a seventh rule means I add one object to a variable; **the resource block itself never changes.**

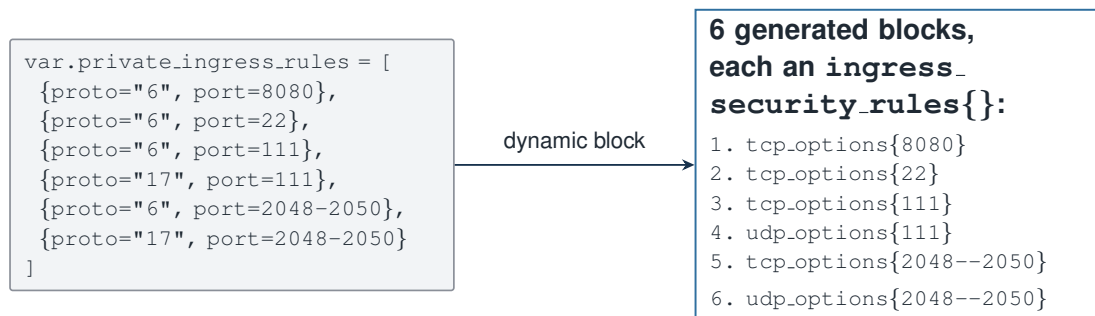


Figure 3.12: One dynamic block, driven by a six-element list, expands into six nested `ingress_security_rules` blocks, each choosing `tcp_options` or `udp_options` by its own protocol value.

3.4 Change-Safety Criteria: Lifecycle Rules and State Refactoring

The `lifecycle` block changes how Terraform treats a resource during planning, rather than declaring anything about the resource itself.

Listing 3.7: `weekTwo/main.tf` – `ignore_changes` on the compute instance

```

1 resource "oci_core_instance" "linux_instance" {
2   # ...
3   lifecycle {
4     ignore_changes = [freeform_tags]
5   }
6 }

```

Without this, every `terraform plan` would show a diff on `freeform_tags` if anything outside Terraform touched the instance’s tags, forcing either a distracting no-op apply on every run or an accidental revert of a legitimate manual change. `ignore_changes` tells Terraform to manage that attribute at creation time only, then stop diffing it. `prevent_destroy = true` takes the *opposite* approach: it makes Terraform refuse to run any plan that would destroy a given resource, including a rename Terraform cannot tell apart from a destroy-and-recreate. That is exactly the failure mode `moved blocks` and `state mv` exist to avoid, and I used both of them, along with `terraform import`, to change this configuration safely.

Beyond lifecycle rules, I relied on three techniques for changing a configuration without destroying infrastructure that did not actually need to change:

- an imperative `terraform state mv` command,
- a declarative `moved` block committed to the repository, and

- `terraform import` for resources that already exist outside Terraform.

Table 3.3 summarizes when each applies.

Table 3.3: Criteria for choosing a change-safety mechanism.

Situation	Mechanism to use
An attribute drifts outside Terraform (e.g. a console-added tag) and should not be reverted on every apply	<code>lifecycle { ignore_changes = [...] }</code>
A resource must never be destroyed by an ordinary plan, even by accident	<code>lifecycle { prevent_destroy = true }</code>
A resource is renamed in code and the fix only needs to reach the person running the command right now	<code>terraform state mv</code>
A resource is renamed in code and every teammate who pulls the repository should get the fix automatically	A committed <code>moved {}</code> block
A resource already exists in OCI but was created outside Terraform	<code>terraform import</code> , or the declarative <code>import</code> block with <code>-generate-config-out</code>

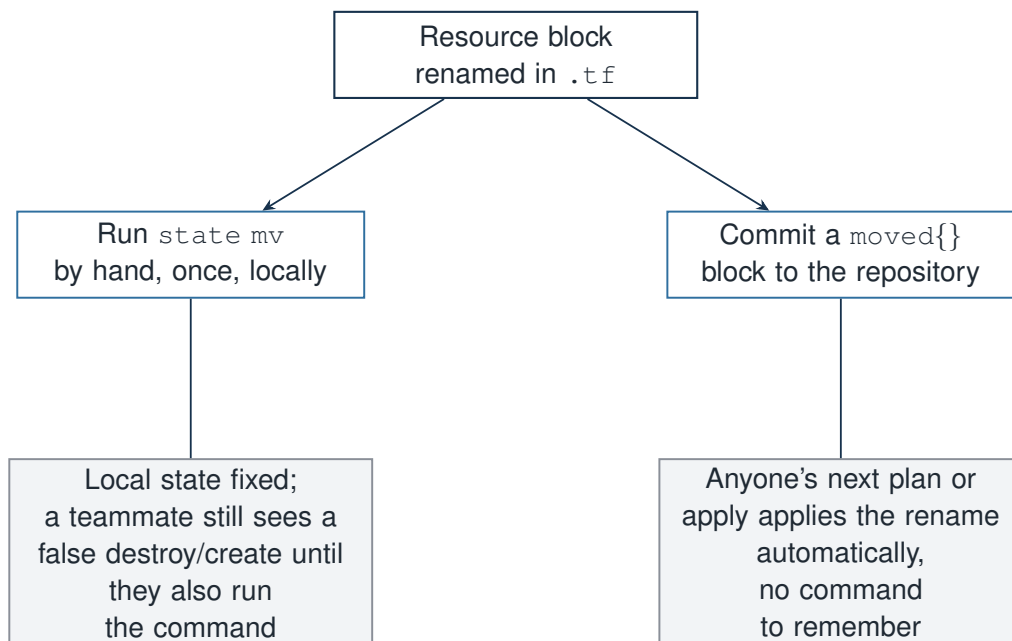


Figure 3.13: Both paths prevent the same spurious destroy-and-recreate; they differ only in who is protected by the fix. Solid outlines mark both paths as implemented and evaluated options, not a specified-versus-implemented distinction.

3.5 State as the Source of Truth

`terraform.tfstate` is a JSON file that is Terraform's own record of every resource it manages: for each one, its Terraform address, its real-world identifier (the OCID OCI assigned it), and every attribute OCI returned about it, not just the ones set in the `.tf` files. This is the only way `terraform plan` can distinguish "this VCN already exists, only the display name changed" from "this VCN does not exist yet, create it."

```

{
  "version": 4,
  "terraform_version": "1.5.x",
  "resources": [
    {
      "type": "oci_core_subnet",
      "name": "tier",
      "instances": [
        { "index_key": "public",
          "attributes": { "id": "ocidl.subnet...",
                        "cidr_block": "10.1.1.0/24", ... } },
        { "index_key": "private", ... }
      ]
    },
    ... one entry per resource ...
  ]
}

```

Figure 3.14: Simplified shape of `terraform.tfstate`. The `index_key` field is how `foreach` addresses (Section 3.3) appear in state, matching each subnet’s own map key.

The state file contains real attribute values in plaintext, including anything OCI returns for resources marked sensitive, which is precisely why I `.gitignore`d it alongside `terraform.tfvars`. **I never hand-edit it**; a typo in raw JSON can desynchronize Terraform’s model from reality in a way that is hard to diagnose. The safe way I inspect it is through the CLI:

Listing 3.8: Safe, read-only ways to inspect state

```

1      state list                                # every resource address
      currently tracked
2      state show oci_core_vcn.terraform_vcn    # full attributes of
      one resource
3      show                                    # human-readable dump of
      the whole state
4  jq '.resources[] | select(.type=="oci_core_instance")' .tfstate
      # scripted queries

```

I used these commands throughout this lab to confirm real values. I read every IP address and OCID quoted in Chapter 4 directly out of `terraform.tfstate` this way, not guessed from the console.

3.6 Assumptions and Limitations

This configuration assumes a **single operator** and a **local state backend**: by default, `terraform.tfstate` is a file on whichever machine last ran `terraform apply`. That works fine for me as one person on a lab exercise. *It breaks the moment a second person needs to run Terraform against the same infrastructure*, because there is no shared source of truth and no locking to prevent two simultaneous applies from corrupting each other's changes. Figure 3.15 contrasts this current setup with the OCI Object Storage backend I already sketched, commented out, in `weekTwo/provider.tf`.



Figure 3.15: Local versus remote (Object Storage) state backend. The dashed node marks the remote backend as specified in `provider.tf` but not currently enabled; the solid node marks the local backend as the implemented, evaluated configuration.

Switching backends is not a change Terraform applies quietly as part of a normal apply. It requires an existing bucket and a deliberate `terraform init -migrate-state` step that copies the current state into it. I left the block commented out to document the intended production pattern without silently changing where my state lives mid-exercise; I carry the recommendation to enable it forward in Section 6.3. I deliberately left two further defaults lab-scoped rather than production-scoped: `bastion_client_cidr_allow_list` defaults to `0.0.0.0/0`, and I left `prevent_destroy` commented out on the VCN so that `terraform destroy` can tear the environment down cleanly at the end of a session. I revisit both as risks in Section 4.5.

3.7 Environment Teardown

At the end of the session I tore the environment down through the OCI console rather than relying on `terraform destroy` alone, since the File Storage stack had accumulated a stray, auto-created mount target and export that never entered Terraform state and so would not have been caught by a plain destroy. I deleted resources in dependency order, matching the order the console itself enforces, so that nothing was left orphaned:

1. File Storage stack (Storage → File Storage).

Deleted the export `/explore-export` first, from the file system's Exports tab, then `explore-mount-target`, then `explore-fs`. In the same pass I found and removed the stray auto-created mount target and export left over from an earlier session (`MountTarget-20260812-2130-49` and its matching `FileSystem-...` export), deleting them in the same export-before-mount-target order.

2. Compute instance.

Compute → Instances → `explore-linux` → Terminate, with the option to also delete the attached boot volume selected so it did not linger as an orphaned resource.

3. Cloud Shell's private network definition.

Deleted `explore-cloudshell-net` from Cloud Shell's Network Configuration panel. This costs nothing left in place, but there was no reason to keep it attached once the lab was over.

4. Subnets.

Networking → `explore-vcn` → Subnets → deleted both `explore-public-subnet` and `explore-private-subnet`.

5. Gateways.

Deleted `explore-nat` and `explore-igw`.

6. Route tables.

Deleted `explore-public-rt` and `explore-private-rt`.

7. Security lists.

Deleted `explore-public-sl` and `explore-private-sl`.

8. VCN.

Deleted `explore-vcn` last. OCI refuses to delete a VCN with anything still attached, so a clean delete here was the final confirmation that every dependency above had actually been removed, and removed in the right order.

Chapter 4

Deployment Results and Verification

In this chapter, I present the environment as I actually deployed and verified it, not as I designed it on paper. Every name, IP address, CIDR block, and command shown below is taken directly from my live Terraform state or from a command I ran against the real environment.

4.1 The Deployed Network Enforces Strict Tier Isolation

Figure 4.1 shows every resource in the environment I deployed: the VCN, both subnets, both gateways, the load balancer and its listener/backend set, the compute instance, File Storage, and the Bastion service, together with the traffic paths between them.

Table 4.1: CIDR allocation for the deployed VCN.

Network	CIDR block	Usable addresses
VCN	10.1.0.0/16	65,536
tf-lab-vcn-AliYousef		
Public subnet tf-lab-public-subnet-AliYousef	10.1.1.0/24	251 (5 reserved by OCI)
Private subnet tf-lab-private-subnet-AliYousef	10.1.2.0/24	251 (5 reserved by OCI)

I sized the VCN at `10.1.0.0/16` (65,536 addresses) and split it into two `/24` subnets, deliberately far more address space than two subnets need, so I can add additional tiers later without renumbering anything already deployed.

Every subnet's ingress and egress rules are generated from `weekTwo/variables.tf` via

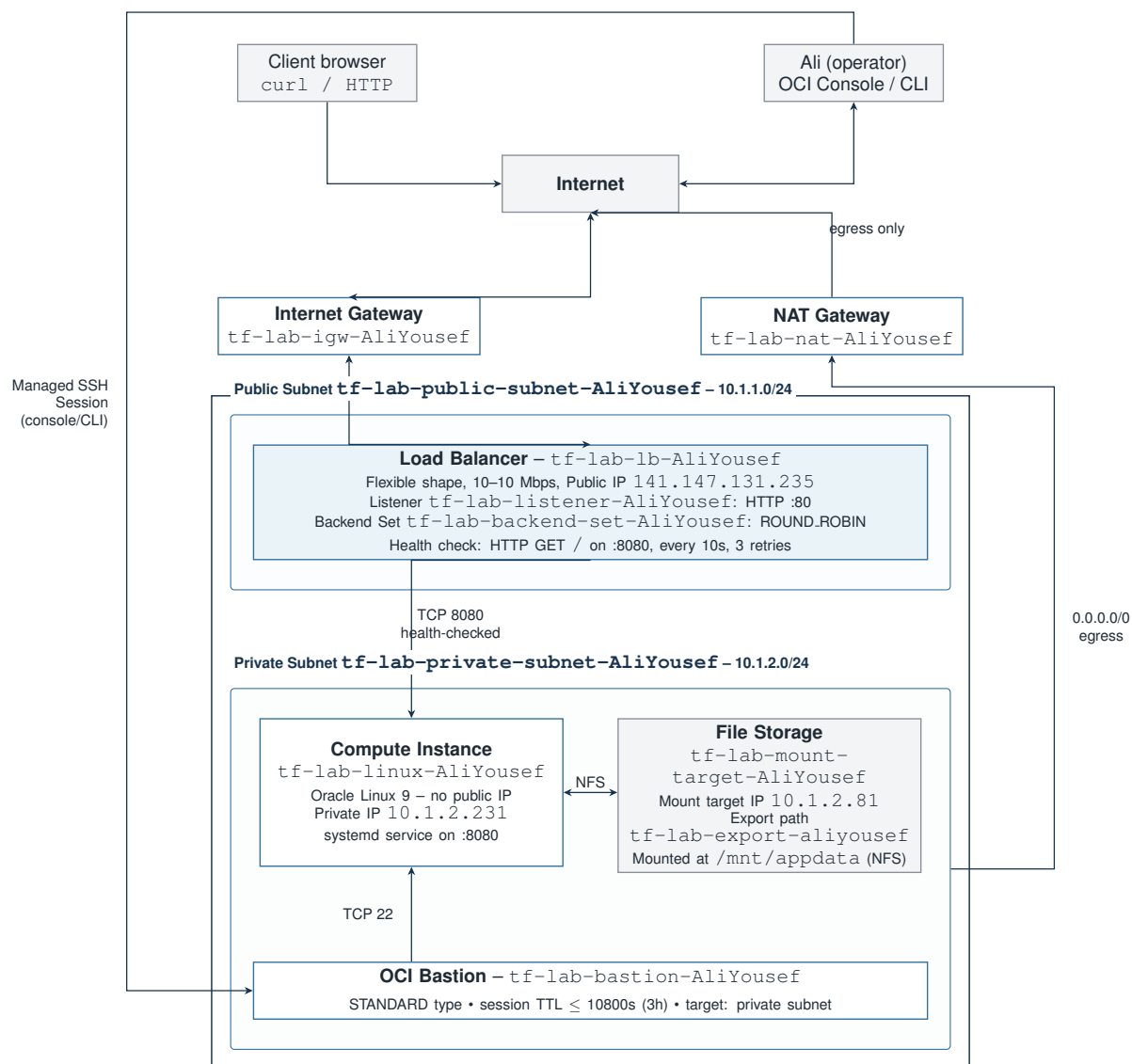


Figure 4.1: Full Week 2 network topology, enclosed in VCN `tf-lab-vcn-AliYousef` (10.1.0.0/16). All names, IPs, and CIDR blocks are taken from live Terraform state; the NAT and Bastion paths are shown as outbound-only or session-scoped rather than permanently open.

the dynamic block I describe in Section 3.3. Table 4.2 lists every rule I actually applied: **this is the authoritative security posture of the environment**, not an aspirational diagram.

Table 4.2: All ingress rules deployed by `weekTwo`. Both lists also egress `0.0.0.0/0` on all protocols.

Security list	Proto/Port	Source	Purpose
web_tier (public)	TCP 80	0.0.0.0/0	Allow HTTP to the load balancer from anywhere
private_tier (private)	TCP 8080	10.1.1.0/24	App traffic, only from the load balancer's subnet
”	TCP 22	10.1.0.0/16	SSH, only from within the VCN, reached via Bastion, never the internet
”	TCP/UDP 111	10.1.0.0/16	NFS portmapper, VCN-internal
”	TCP/UDP 2048–2050	10.1.0.0/16	NFS/mount protocol ports, VCN-internal

The pattern to notice: **the private subnet's inbound rules that originate outside the VCN number zero**. Every rule sources from either the public subnet's own `/24` or the VCN's own `/16`. *Nothing in the private subnet is reachable from the public internet under any circumstance*, which is exactly the property a private application tier is supposed to have.

Table 4.3: Route tables. The private route still points `0.0.0.0/0` at the NAT gateway, giving outbound-only internet access without ever accepting an inbound connection, because NAT gateways are stateful and egress-only by design.

Route table	Destination	Target
Public (attached to public subnet)	0.0.0.0/0	Internet Gateway
Private (attached to private subnet)	0.0.0.0/0	NAT Gateway

Figure 4.2 is the OCI Console's VCN details page for my live environment, confirming the CIDR block, DNS label, and compartment match my Terraform configuration exactly.

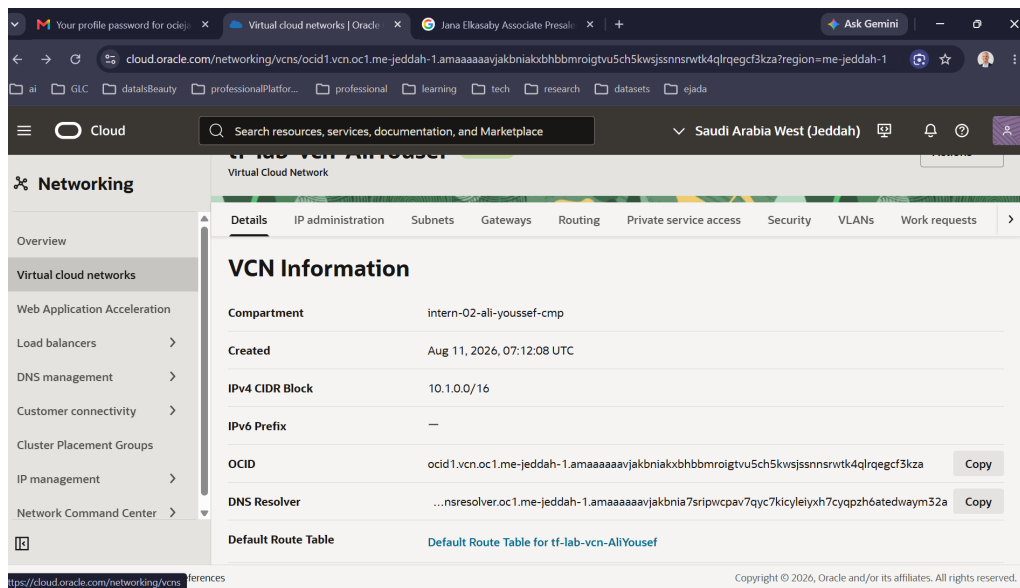


Figure 4.2: OCI Console: `tf-lab-vcn-AliYousef`, CIDR `10.1.0.0/16`, compartment `intern-02-ali-youssef-cmp`, region `me-jeddah-1`.

4.2 The Load Balancer Health-Checks and Routes Traffic End-to-End, Verified Live

Listing 4.1: Listener, backend set, and health checker, from `weekTwo/loadbalancer.tf`

```

1 resource "oci_load_balancer_backend_set" "app_backend_set" {
2   name           = "${local.name_prefix}-backend-set-${local.owner}"
3   load_balancer_id = oci_load_balancer_load_balancer.app_lb.id
4   policy         = "ROUND_ROBIN"
5
6   health_checker {
7     protocol      = "HTTP"
8     port          = var.app_port
9     url_path      = "/"
10    return_code    = 200
11    interval_ms    = 10000
12    timeout_in_millis = 3000
13    retries        = 3
14  }
15 }
16
17 resource "oci_load_balancer_backend" "app_backend" {
18   load_balancer_id = oci_load_balancer_load_balancer.app_lb.id
19   backendset_name  = oci_load_balancer_backend_set.app_backend_set.name
20   ip_address       = oci_core_instance.linux_instance.private_ip
21   port             = var.app_port

```

```
22 }
23
24 resource "oci_load_balancer_listener" "app_listener" {
25   load_balancer_id      = oci_load_balancer_load_balancer.app_lb.id
26   name                  = "${local.name_prefix}-listener-${local.owner
    }"
27   default_backend_set_name = oci_load_balancer_backend_set.
    app_backend_set.name
28   port                  = 80
29   protocol              = "HTTP"
30 }
```

I made three details in this configuration **deliberate design choices, not arbitrary defaults**:

- **Private-IP registration.** The backend is registered by the instance's private IP because the instance has no public IP at all; Terraform's implicit dependency graph guarantees the instance is created before this backend reads that IP.
- **Real HTTP health checks.** The health checker makes a real HTTP GET against the application rather than just a TCP handshake, catching failures a TCP-level check would miss.
- **Tolerant retry timing.** A backend must fail three consecutive checks (30 seconds) before being marked Critical, and pass one check to be marked OK again.

That last point was not theoretical for me: during my own deployment, the backend showed Critical for the first couple of minutes after `terraform apply` finished, because `cloud-init` was still retrying the NFS mount before the `systemd` service could start. *That is exactly the scenario this health check design exists to catch and recover from automatically.*

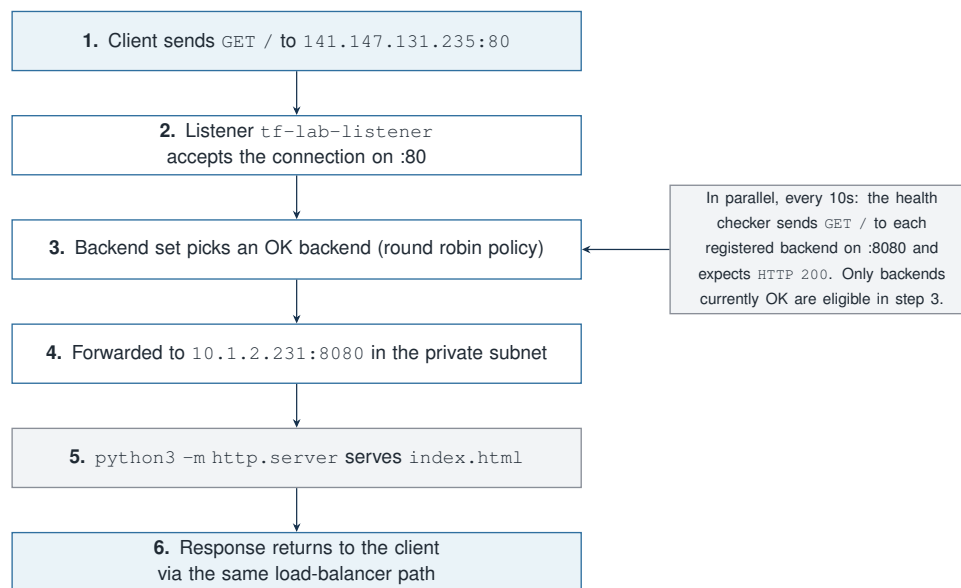


Figure 4.3: Request path for a single HTTP request, and the parallel, continuous health-check loop that decides which backends are eligible to receive it. All steps are implemented and verified against the live environment.

I verified this exact path live: running `curl http://141.147.131.235` from outside OCI returned the application’s HTML (“*Ejada Internship - Week 2 Lab*”), confirming every hop in Figure 4.3 actually works, not just that the resources exist. Figures 4.4–4.7 are the OCI Console evidence I captured.

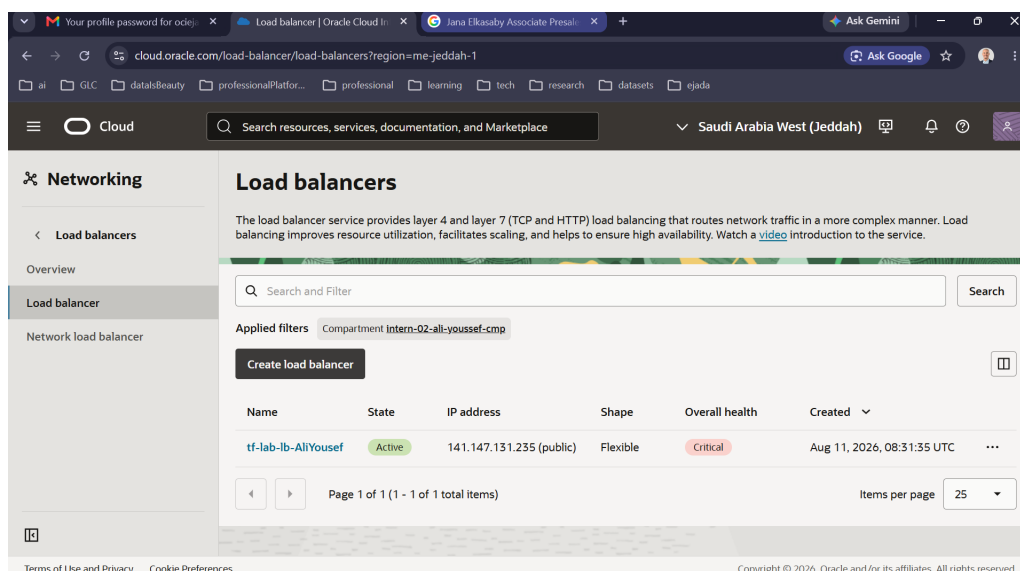


Figure 4.4: The load balancer, Active, with its public IP 141.147.131.235.

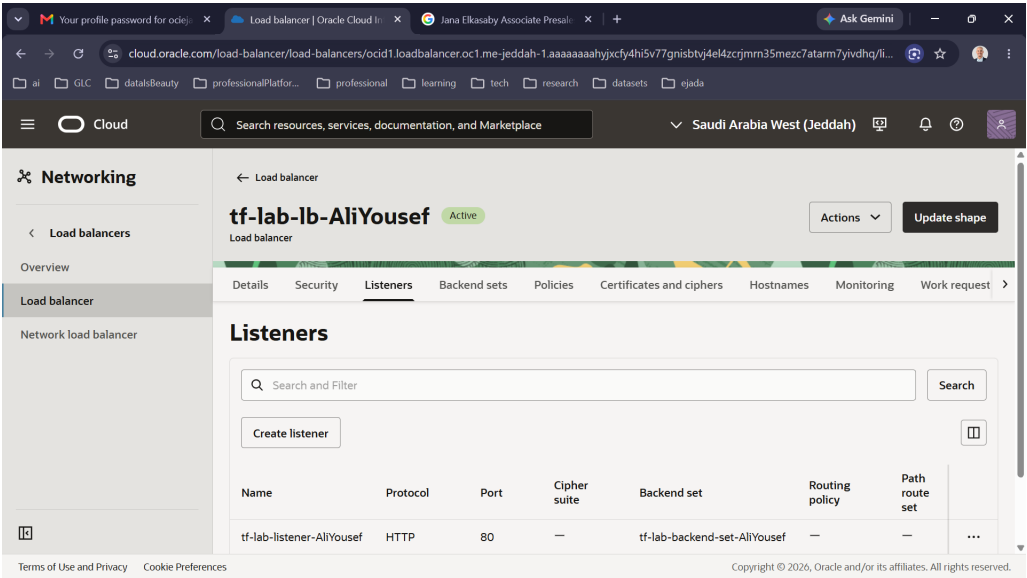


Figure 4.5: Listener `tf-lab-listener-AliYousef`: HTTP on port 80, forwarding to backend set `tf-lab-backend-set-AliYousef`.

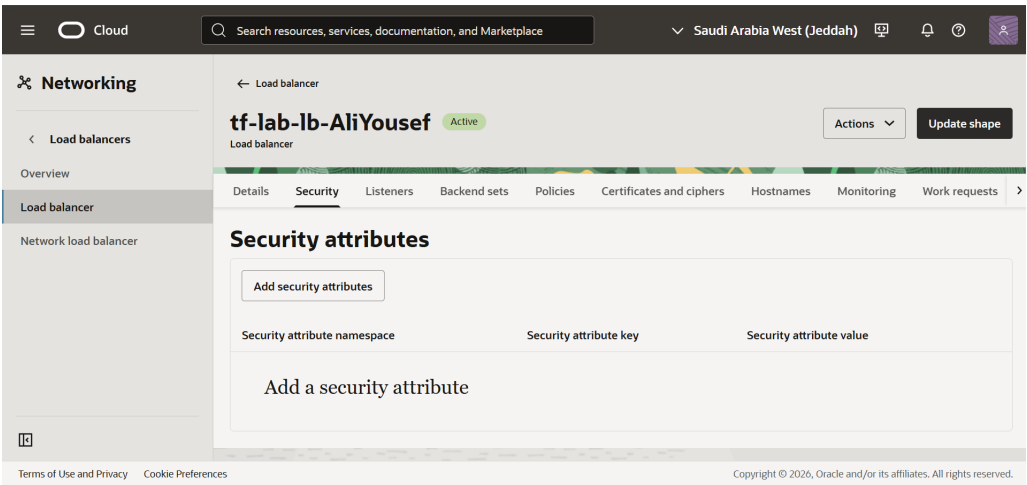


Figure 4.6: Backend set with its round-robin policy and one registered backend.

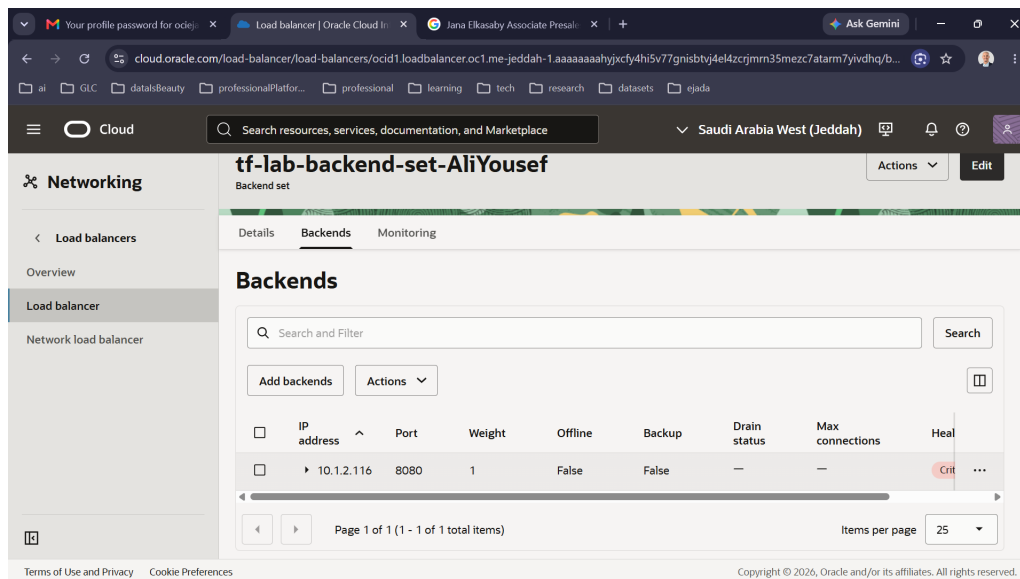


Figure 4.7: The individual backend entry: private IP and application port, captured mid-deployment while the health check was still converging.

4.3 Private Instance Access Is Ephemeral, Audited, and Requires No Standing Attack Surface

I used the **OCI Bastion Service** in this build, choosing it over the alternatives compared in Table 2.4 for three reasons:

- **No extra infrastructure to secure.** A jump VM is itself a server that needs patching, its own security list, and its own monitoring, effectively doubling the attack surface being defended; Bastion is a managed control plane with no VM to operate.
- **No standing attack surface.** Every session is created on demand, expires automatically (capped by OCI at 10800 seconds, or 3 hours), and is logged, so there is no standing SSH session and no permanently open port sitting idle between uses.
- **The target's security posture never changes.** The private instance's security list allows TCP 22 only from 10.1.0.0/16, and Bastion sessions originate from inside OCI's control plane and reach the target over the VCN, so this VCN-only rule is sufficient.

Listing 4.2: weekTwo/bastion.tf – the Bastion resource itself

```
1 resource "oci_bastion_bastion" "tf_lab_bastion" {
2   compartment_id = local.my_compartment_id
3   bastion_type   = "STANDARD"
4   target_subnet_id = oci_core_subnet.tier["private"].id
5   name           = "${local.name_prefix}-bastion-${local.
    owner}"
```

```
6 client_cidr_block_allow_list = var.bastion_client_cidr_allow_list
7 max_session_ttl_in_seconds   = var.bastion_max_session_ttl_in_seconds
8 }
```

Note what I left out of this resource block: there is no per-connection session defined in Terraform. That is intentional. The Bastion service is long-lived infrastructure and belongs in Terraform, but an individual session is meant to be created on demand, not kept as a permanent resource that would need constant re-creation as it expires. I create sessions via the console or CLI: navigate to Identity & Security → Bastion → `tf-lab-bastion-AliYousef`, choose Create Session → Managed SSH Session, target the compute instance's OCID (available from Terraform's `instance_id` output) on port 22, and supply the SSH public key matching the private key I already used in the instance's `ssh_authorized_keys`. OCI returns an SSH command that tunnels through the Bastion service to the instance, valid until the session's TTL elapses.

The variable `bastion_client_cidr_allow_list` controls which source IPs may even open a session against this Bastion. This is a different, additional control from the security-list rule in Table 4.2: the security list controls what the target instance accepts, while this allow list controls who can talk to the Bastion service itself. I examine its current default as a risk in Section 4.5.

4.4 State Refactoring and Import, Demonstrated Live

Section 3.4 explained what `terraform state mv`, the moved block, and `terraform import` are for in general. In this section, I exercise each one against the real environment.

I renamed the public-tier security list in code from `public.security_list` to `web_tier.security_list`. Before I ran any command, `terraform plan` showed OCI's security list would be destroyed and a new one created, identical in every attribute, differing only in the Terraform-internal name.

Listing 4.3: The command actually run to fix it

```
1 state mv \
2 oci_core_security_list.public_security_list \
3 oci_core_security_list.web_tier_security_list
```

After this, **`terraform plan` showed no changes at all**: state now says the existing security list *is* `web_tier.security_list`, matching the renamed block in `main.tf`, and nothing needed to happen against the real OCI API. For the equivalent private-tier rename (`private.security_list` → `private_tier.security_list`), I used a committed moved block instead, so I have both approaches side by side in `weekTwo/` for direct comparison:

Listing 4.4: The complete weekTwo/moved.tf

```
1 moved {
2   from = oci_core_security_list.private_security_list
3   to   = oci_core_security_list.private_tier_security_list
4 }
```

Both renames produced the same outcome, an unchanged plan, but the `moved` block leaves that outcome committed to the repository rather than dependent on a command someone remembers to run.

I then created an NSG directly in the OCI Console, outside Terraform entirely, to simulate infrastructure created by hand before anyone decided to manage it as code. My file `weekTwo/import_demo.tf` declares a resource block describing it:

Listing 4.5: The complete weekTwo/import_demo.tf

```
1 resource "oci_core_network_security_group" "imported_nsg" {
2   compartment_id = local.my_compartment_id
3   vcn_id         = oci_core_vcn.terraform_vcn.id
4   display_name   = "tf-lab-imported-nsg-AliYousef"
5 }
```

Writing this block does not create anything by itself. Until I linked it to the real OCID with an explicit `import`, `terraform plan` would try to create a second, duplicate NSG with the same name.

Listing 4.6: The import command actually run

```
1         import oci_core_network_security_group.imported_nsg \
2   ocid1.networksecuritygroup.oc1.me-jeddah-1.
         aaaaaaaawj4t3u3jeb4yyce7oy5jxc5274qxh4xmyvp2qrvpafkjjzix2a4yq
```

After this, `terraform state show` on the resource returned the real NSG's attributes, and `terraform plan` showed no changes: the resource block's arguments already matched what I had created by hand in the console.

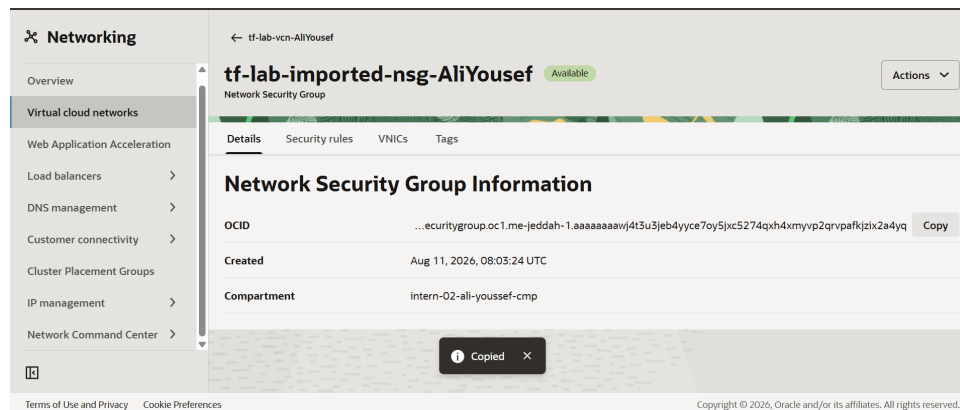


Figure 4.8: The manually created NSG (`tf-lab-imported-nsg-AliYousef`) in the OCI Console, prior to being brought under Terraform management with the command above.

Terraform 1.5+ also offers a declarative `import` block, parallel to how `moved` supersedes `state mv`. It can be combined with the flag `-generate-config-out=generated.tf` on `terraform plan` to have Terraform write the resource block automatically. I used the manual `terraform import` command deliberately, so the import step itself stayed a visible, explicit action rather than something folded into a routine plan.

4.5 Risk Analysis

Table 4.4: Risks identified in the current configuration.

Risk	Likelihood	Impact	Mitigation or Monitoring Indicator
<code>bastion_client_cidr_allow_list</code> defaults to <code>0.0.0.0/0</code>	High	Medium	Bastion sessions still require a valid SSH key, but the allow list should be narrowed to known operator addresses before longer-lived use; see Section 6.3
Local Terraform state, no locking	Medium	High	Only matters once a second operator is involved, but concurrent applies could then corrupt state; migrate to the OCI Object Storage backend already specified in <code>provider.tf</code>
<code>prevent_destroy</code> left off the VCN	Low	High	Low likelihood in this lab, since destroy is intentional at session end; add <code>prevent_destroy</code> before any non-lab reuse
Health-check convergence delay after apply	Low	Low	Observed once and self-resolving; already mitigated by the retry/interval design in Section 4.2

4.6 Key Unknowns

Several questions remain open for me because they fall outside this lab's scope, not because evidence is missing. I have not exercised the build under concurrent multi-operator Terraform runs, so I have identified the local-state risk above without observing it in practice. I ran no load or performance testing against the load balancer or the application, so I do not know how it behaves above the lab-scale 10 Mbps flexible bandwidth ceiling. My listener is HTTP only; I do not address here whether or how I would add an HTTPS listener with a managed certificate. Finally, I did not exercise the modern `import` block with `-generate-config-out`, so I do not know how closely its generated configuration would match the hand-written `import_demo.tf` I used in this lab.

Chapter 5

Conclusions

5.1 Main Conclusion

The environment I built and documented here implements a standard **three-path defense** for a private application tier: a public load balancer as the only internet-facing surface, a private instance reachable only through that load balancer for application traffic and only through a managed, audited Bastion for administrative traffic, and outbound-only NAT for the private tier's own update traffic. Every rule in Table 4.2 traces back to one of those three paths, and *no other path exists*. That answers the objective I set out in Section 1.2 directly: **the network I deployed enforces the three-tier defense model it was designed around**, and I have shown, live, that the Terraform configuration I built it with is safely refactorable without destroying resources that did not need to change.

5.2 Strongest Supporting Evidence

Three pieces of evidence carry the most weight for me:

- **The live `curl` test** I ran against the load balancer's public IP returned the application's actual HTML, confirming the whole path in Figure 4.3 works end to end rather than merely existing as configured resources.
- **Zero-change state refactors.** `terraform plan` showed zero changes after both the `state mv` rename and the `moved-block` rename I performed in Section 4.4, confirming the refactor techniques worked exactly as designed, not just that the commands ran without error.
- **The literal, deployed security list.** Table 4.2 shows every inbound rule on the private tier sourcing from inside the VCN: the actual Terraform-generated ruleset, not an aspirational

diagram of intended behavior.

5.3 Biggest Uncertainty

The largest uncertainty for me is how the configuration would behave with more than one operator or under real traffic, because *I have not tested either*. The local, unlocked state backend (Section 3.6) is a known risk in theory but has not caused a conflict in practice, since I have only ever run Terraform against it myself, as a single operator. Reducing that uncertainty would mean either enabling the remote backend and deliberately testing a concurrent apply, or accepting the risk as out of scope for a two-week individual lab exercise.

5.4 Strategic Implications and Recommendation

I judge the current configuration appropriate as-is for continued use by a single operator in a training context. Before I would reuse this pattern for a team or a production workload, I would make three changes, each already specified in the configuration but left disabled for this lab:

- **migrate state** to the OCI Object Storage backend;
- **narrow the Bastion client CIDR allow list** to known operator addresses; and
- **add prevent_destroy** to any resource that must never disappear by accident.

I carry these forward as concrete actions in Section 6.3.

5.5 Confidence Level and Final Judgement

My confidence in the findings of Chapter 4 is **high**: I backed every claim about the deployed environment with the live Terraform state, an OCI Console screenshot, or a command I executed against the real infrastructure, following the evidentiary basis I set out in Section 2.4. My confidence in the forward-looking risk assessment is **medium**: the risks I identify in Section 4.5 follow directly from the configuration as written, but I have *not* observed their real-world impact, state corruption under concurrency or exposure from the open Bastion allow list. I have only reasoned about them from the configuration's current defaults.

Chapter 6

Recommendations and Next Steps

6.1 Further Testing Needed

Two follow-up exercises would materially reduce the uncertainties I named in Section 5.3:

- a deliberate test of a **concurrent `terraform apply`** from two operators against a shared backend; and
- a basic **load test** against the load balancer's public endpoint, to see how it behaves as traffic approaches the configured 10 Mbps flexible-shape ceiling.

6.2 Data to Validate

I verified everything covered in Chapter 4 against the live environment at the time of writing. **The item most likely to go stale first is the Bastion session TTL and allow-list configuration**, since these are exactly the settings I recommend changing in Section 6.3. If I, or anyone else, reuse this configuration later, I would re-read `variables.tf` directly rather than trust the defaults quoted here.

6.3 Recommended Actions

Before I reuse this pattern for a team or a production workload, here is what I would do first:

1. **Enable the remote state backend.**

Uncomment the `backend "oci"` block in `provider.tf`, create the target Object Storage bucket, and run `terraform init -migrate-state` to move the existing local state into it before any second operator touches this configuration.

2. Narrow the Bastion allow list.

Change `bastion_client_cidr_allow_list` from `0.0.0.0/0` to the specific `/32` addresses of operators who actually need to open sessions.

3. **Add a genuine database tier.** The subnet design already reserves address space for one. Adding it is a matter of extending `var.subnet_tiers` and the corresponding ingress rules, not renumbering anything that exists today.
4. **Add an HTTPS listener.** The current listener is HTTP only. A production-facing version of this build would add a second listener on 443 with a managed certificate.
5. **Re-enable `prevent_destroy`** on the VCN and any other resource that must not disappear by accident, once the environment is no longer torn down at the end of every session.

6.4 Monitoring Indicators

Table 6.1: Indicators worth monitoring if this environment moves beyond a single-operator lab.

Indicator	Why It Matters	Review Frequency
Backend health status (OK vs. Critical)	Directly shows whether the application tier is reachable and serving correctly	Continuous, via the existing 10s health check
Bastion active session count and source IPs	Confirms only expected operators are opening sessions once the allow list is narrowed	Weekly, or on every review of <code>bastion_client_cidr_allow_list</code>
Load balancer bandwidth utilization vs. the 10 Mbps flexible ceiling	Signals when the bandwidth range needs to be raised	Before and after any load test

6.5 Open Questions

Three questions I would answer before treating this configuration as a template for other builds:

- whether the modern declarative `import` block with `-generate-config-out` produces configuration close enough to hand-written `import_demo.tf` to replace the manual workflow;
- whether the current single-AD design should become multi-AD once a database tier is added; and
- at what point the `bastion_max_session_ttl_in_seconds` default of 10800 seconds should be shortened for a non-lab environment.

Appendix

.1 Glossary

Table 2: Terms used throughout this documentation.

Term	Meaning
VCN	Virtual Cloud Network: an isolated, software-defined network in an OCI region; the top-level container for subnets, gateways, and route tables.
CIDR block	A range of IP addresses expressed as <code>address/prefix-length</code> , e.g. <code>10.1.0.0/16</code> .
Subnet	A subdivision of a VCN's CIDR block; every resource with a private IP lives in exactly one subnet.
Route table	A set of rules mapping destination CIDR blocks to a target (gateway); attached to a subnet.
Security list	A stateful or stateless firewall attached to a subnet, applying to every resource in it.
NSG	Network Security Group: like a security list, but attached directly to individual resources (VNICs) rather than a whole subnet.
Internet Gateway (IGW)	Gives a subnet two-way connectivity to the public internet.
NAT Gateway	Gives a subnet outbound-only internet connectivity; nothing from the internet can initiate a connection through it.
Listener	The load balancer object a client connects to (protocol + port).
Backend set	A named pool of backends plus a load-balancing policy and health checker.
Health checker	The probe that marks each backend OK or Critical.
Bastion	OCI's managed service for brokering short-lived, audited SSH sessions to resources with no public IP.
Variable	A Terraform input, supplied externally (person, <code>tfvars</code> file, CI pipeline).
Local	A Terraform value computed once inside the configuration from other available values.
State file	<code>terraform.tfstate</code> ; Terraform's own record of every resource it manages and its real-world attributes.

.2 Command Reference

Table 3: Terraform commands referenced in this documentation, all run against `weekTwo/` during my work on this lab.

Command	Used for
<code>terraform state list</code>	List every resource address currently tracked in state
<code>terraform state show addr</code>	Show full attributes of one resource from state
<code>terraform state mv old new</code>	Rename a resource's address in state without touching real infrastructure
<code>terraform import addr id</code>	Bring an existing real resource under Terraform management
<code>terraform plan -generate- config-out=file</code>	Modern alternative to manual import: have Terraform write the resource block
<code>terraform init -migrate-state</code>	One-time step required when switching state backends
<code>terraform fmt -check</code>	Verify consistent formatting without modifying files
<code>terraform validate</code>	Catch configuration or syntax errors before planning

References

This documentation covers one specific deployment I built and verified live, rather than a literature-based analysis. I did not cite external sources, third-party claims, or published works. Every factual claim is instead traceable to my live Terraform state, an OCI Console screenshot I captured, or a command I executed directly against the deployed environment, following the evidentiary basis I set out in Section 2.4. I kept `References/myref.bib` in the repository for template compatibility, but it intentionally contains no entries, so no citation keys appear in this documentation.